

Hallucination / Failure Scenarios

Three realistic failure modes for this system, each grounded in the actual sample transcript used during development and, where applicable, an incident that genuinely occurred while building this prototype (Scenario 2). For each: description, an example transcript excerpt, what incorrect LLM behavior would look like, the impact if it reached a coach unfiltered, what this implementation currently does about it, and what a production version should add.

Scenario 1: Hallucinated / Unsupported Claim

Description

The model states something about the client that sounds plausible and consistent with the general tone of the conversation, but is not actually supported by anything said in the transcript — e.g. inferring a medical cause, a trend, or a quantified detail that was never stated.

Example transcript excerpt

```
Day 5
Client: Weight seems slightly up even though I'm eating almost half of what I used
to eat.
Coach: It is not always about eating less. Your body needs adequate nutrition.
```

Incorrect LLM behavior

The model writes something like: "*Client's weight gain is likely due to a slowed metabolism from prolonged undereating, indicating a possible metabolic adaptation risk,*" and tags it `confirmed_fact` — none of this is stated anywhere in the transcript; it's a plausible-sounding medical explanation the model generated from general knowledge, not from the conversation.

Impact

A coach could treat this as an established fact about the client's physiology and adjust the nutrition plan around it, when the transcript actually contains no such information — the client only reported a subjective impression of gaining weight.

Current mitigation in this implementation

- The prompt explicitly forbids stating anything not traceable to a quoted line, and forbids diagnosing or assigning medical causes at all.
- Every non-missing `GradedField` in the Pydantic schema **requires** at least one `evidence` item with a real quote — this specific fabricated claim would need a quote from the transcript to pass validation, and none exists, so if the model tried to cite something, the citation itself would either be fabricated (and therefore not string-verifiable against the source in a future grounding check) or the field would fail the "evidence required" validator and be rejected.

- Confidence banding pushes weak/unsupported inferences toward `missing_information` rather than a confident-sounding claim.

Future improvement

Add a deterministic **grounding check**: after parsing, verify that each `evidence.quote` is an actual substring (or a close fuzzy match) of the corresponding day's transcript text. Any evidence item that doesn't verify against the source would cause that specific field to be rejected and retried, closing the gap where a model could satisfy the schema by citing a *fabricated* quote rather than a real one.

Scenario 2: Invalid / Malformed JSON Output (Response Truncation)

Description

Gemini returns a response that is not valid JSON — most commonly because it was cut off mid-object before the closing braces were written. This actually occurred during development: `gemini-3.5-flash` consistently returned syntactically invalid, truncated JSON (failing at roughly the 272nd character) across multiple retries.

Example transcript excerpt

Any transcript of realistic length triggers this risk — the failure is about output generation, not input content. The 8-day sample transcript used throughout development was sufficient to trigger it.

Incorrect LLM behavior

The response is cut off partway through, e.g.:

```
{
  "client_id": "anonymized_client",
  "report_id": "report_abc123",
  "period": { "start_day": "Day 1", "end_day":
```

`json.loads()` fails with `Expecting ',' delimiter: line 17 column 4`.

Impact

Without handling, the frontend would either crash trying to parse the response or silently show nothing, with no indication of what went wrong — the coach would have no report and no explanation.

Current mitigation in this implementation

- **Root cause identified and fixed at the source:** Gemini's internal "thinking" tokens were consuming most of `max_output_tokens` before the model wrote the actual JSON, truncating it. `thinking_config` is now set to disable thinking for this task (with a one-time fallback if a model rejects that parameter), and `LLM_MAX_TOKENS` was raised as a second, independent safety margin.
- **Two independent retry layers** remain as a backstop regardless of cause: `llm_service.py` retries the raw Gemini call once internally if the response isn't valid JSON, and `analysis_service.py` retries up

to `MAX_LLM_RETRIES` (default 2) additional times with a correction note describing the exact parse error.

- **Markdown fence stripping** at two layers, in case Gemini wraps output in ````json` despite `response_mime_type=application/json`.
- **No silent failure**: if every retry is exhausted, the endpoint returns `502` with a descriptive error — never a `200` with partial or corrupted data, and never a raw traceback.
- **Diagnostic logging**: every call logs Gemini's token usage (prompt/thinking/output/total) and explicitly flags when a response was cut off by the token limit, so this failure mode is visible in logs rather than only inferable from a JSON parse error position.

Future improvement

Use Gemini's `response_schema` parameter (structured output mode) in addition to `response_mime_type="application/json"`, which constrains generation to match a provided JSON schema token-by-token rather than relying on prompt instructions and post-hoc validation alone — this would reduce both truncation risk and schema-violation risk at the generation step itself, rather than catching them only after the fact.

Scenario 3: Conflicting Statements Within the Transcript

Description

The client says two things in the same conversation that don't agree with each other — common in real coaching check-ins where someone answers a quick question dismissively and elaborates later, or under/over-reports depending on mood.

Example transcript excerpt

```
Day 3
Coach: Did you have salad before lunch?
Client: No. I still need to stock vegetables properly. Will do it tomorrow.
...
Client: Lunch had lots of vegetables, curd and some protein.
```

(The client says "no" to a specific food question, then later describes a lunch that sounds like it included vegetables.)

Incorrect LLM behavior

The model silently picks one version and states it as settled fact — e.g. reporting nutrition adherence as strong because "lunch had lots of vegetables" while ignoring the earlier "no" answer, or vice versa — without flagging that the two statements don't fully agree.

Impact

The coach loses visibility into a genuine ambiguity in the client's self-reporting. Nutrition adherence could be assessed as better or worse than it actually is, and a real conversation point (asking the client to clarify) is lost

because the report presents false certainty.

Current mitigation in this implementation

- The prompt explicitly instructs: *"If two messages contradict each other, do NOT silently pick one. Report both, tag as **conflicting_reports**, and let a human resolve it."*
- The schema has a dedicated **conflicting_reports** array, requiring a **minimum of two evidence items** — enforced by Pydantic — so a conflict must be shown as two sides, not collapsed into one.
- These conflicts render as their own section in the frontend (**ReportBody.jsx**), separate from the main category cards, so they aren't buried inside a normal nutrition/exercise/etc. finding.

Future improvement

Add a lightweight deterministic pre-pass over same-day messages before the LLM call — flagging pairs of statements that are lexically close to direct contradictions (e.g. a "no" followed later by a positive description on the same topic) — and pass those flagged pairs to the model as explicit "check these for conflict" hints, rather than relying entirely on the model to notice contradictions unprompted across a long transcript.